

Friedrich-Alexander-Universität
Erlangen-Nürnberg (FAU)

NHR@FAU

Assignment 5: Multicore power and energy

Programming Techniques for Supercomputers (PTfS)

Muhammad Omar
Student IdM: to37gulo
Student HPC-Account: ptfs356v

Gehan Younes
Student IdM: qi39dyry

Ahmed Mohamed
Student IdM: gi10dafy

June 11, 2026

Q1: Multicore power envelope (30 crd)

Model: a single core at base frequency f_0 dissipates $W = W_d$, and $W \propto f^3$, hence

$$W(f) = W_d \left(\frac{f}{f_0}\right)^3.$$

(a) Overclocking by 30% (5 crd)

With $f = 1.3\, f_0$ (e.g. 2.0 → 2.6 GHz):

$$\boxed{W = W_d(1.3)^3 = 2.197\, W_d \approx 2.2\, W_d.}$$

Power rises by a factor ≈ 2.2 (+120%) for a 30% clock increase – the cubic law makes overclocking very expensive in power.

(b) Trading clock for cores at constant power (10 crd)

Put m cores at reduced frequency f and require the total power to stay at W_d:

$$m\, W_d \left(\frac{f}{f_0}\right)^3 = W_d \quad \Rightarrow \quad \frac{f}{f_0} = m^{-1/3}.$$

The required relative clock reduction is therefore

$$\boxed{\frac{\Delta f}{f_0} = 1 - m^{-1/3}.}$$

Cutting the clock in half, $f/f_0 = \frac{1}{2}$:

$$m = \left(\frac{f_0}{f}\right)^3 = 2^3 = \boxed{8 \text{ cores.}}$$

(c) Performance gain for $m = 8$ (5 crd)

Eight cores at $f_0/2$ versus one core at f_0, same power, same memory interface:

- **Maximum gain** = 4× – for perfectly scalable, compute-bound code that lives on *parallel* (private) resources. Performance ∝ cores × clock: $\dfrac{8 \cdot (f_0/2)}{1 \cdot f_0} = 4$.
- **Minimum gain** = 1× – for memory-bound code that saturates the *shared* memory interface. The bus is unchanged and already saturated, so the eight slower cores buy nothing.

The maximum is reached by core-bound (cache/register-resident) code; the minimum by bandwidth-bound code limited by the shared memory bus.

(d) Optimal frequency for minimum energy (10 crd)

For a compute-bound workload the runtime scales as $T \propto 1/f$, so

$$E = W \cdot T \propto f^3 \cdot f^{-1} = f^2.$$

Since $E \propto f^2$ is minimised by the slowest possible clock, $\boxed{f_{\text{opt}} = f_{\min}}$. (This crude conclusion is refined in Q2(d), where a non-zero baseline power moves the optimum.)

Q2: Optimal energy to solution (60 crd)

Refined model for an n-core processor:

$$W(n) = W_0 + n\,W_d, \qquad P(n) = \min\big(n\,P(1),\, P_{\text{limit}}\big), \qquad T(n) = \frac{1}{P(n)},$$

$$\boxed{E(n) = W(n)\,T(n) = \frac{W_0 + n\,W_d}{\min\big(n\,P(1),\, P_{\text{limit}}\big)}.}$$

The energy minimum sits at the saturation knee $n_{\min} = P_{\text{limit}}/P(1)$ (when $n_{\min} \le 52$).

(a) Baseline: $W_0 = 120$, $W_d = 4$, $P(1) = 1$, $P_{\text{limit}} = 20$ (15 crd)

$$E(n) = \begin{cases} \dfrac{120 + 4n}{n} = \dfrac{120}{n} + 4, & n \le 20, \\ \dfrac{120 + 4n}{20} = 6 + 0.2\,n, & n \ge 20. \end{cases}$$

Decreasing then increasing $\Rightarrow$ $\boxed{n_{\min} = 20, \quad E_{\min} = 10 \text{ J}.}$

More than $n_{\min}$ cores? No. Beyond saturation $T = 1/P_{\text{limit}}$ is constant while W keeps rising, so energy only increases. Using more than $n_{\min}$ cores wastes energy without any speedup.

(b) SIMD optimisation: $P(1) = 2.5$, $P_{\text{limit}} = 20$ (15 crd)

Saturation at $n = 20/2.5 = 8$:

$$\boxed{n_{\min} = 8, \quad E_{\min} = E(8) = \frac{48}{8} + 1.6 = 7.6 \text{ J}.}$$

Conclusion: improving single-core performance lowers *both* the optimal core count ($20 \to 8$) and the minimum energy ($10 \to 7.6$ J): faster scalar code reaches the bottleneck sooner and finishes more cheaply.

(c) Raising P_{limit}, keeping $P(1) = 2.5$ (15 crd)

$P_{limit} = 30$*:* saturation at $n = 12$,

$$\boxed{n_{\min} = 12, \quad E_{\min} = \frac{48}{12} + 1.6 = 5.6 \text{ J}.}$$

$P_{limit} = 200$*:* saturation would require $n = 80 > 52$ cores, so the code *never* saturates. Then $E(n) = 48/n + 1.6$ falls monotonically and the optimum is at the maximum core count:

$$\boxed{n_{\min} = 52, \quad E_{\min} = E(52) = \frac{48}{52} + 1.6 \approx 2.52 \text{ J}.}$$

Conclusion: raising the saturation limit pushes the energy optimum to higher core counts and lowers energy. If the bottleneck is removed entirely, the code is scalable and one should use *all* available cores.

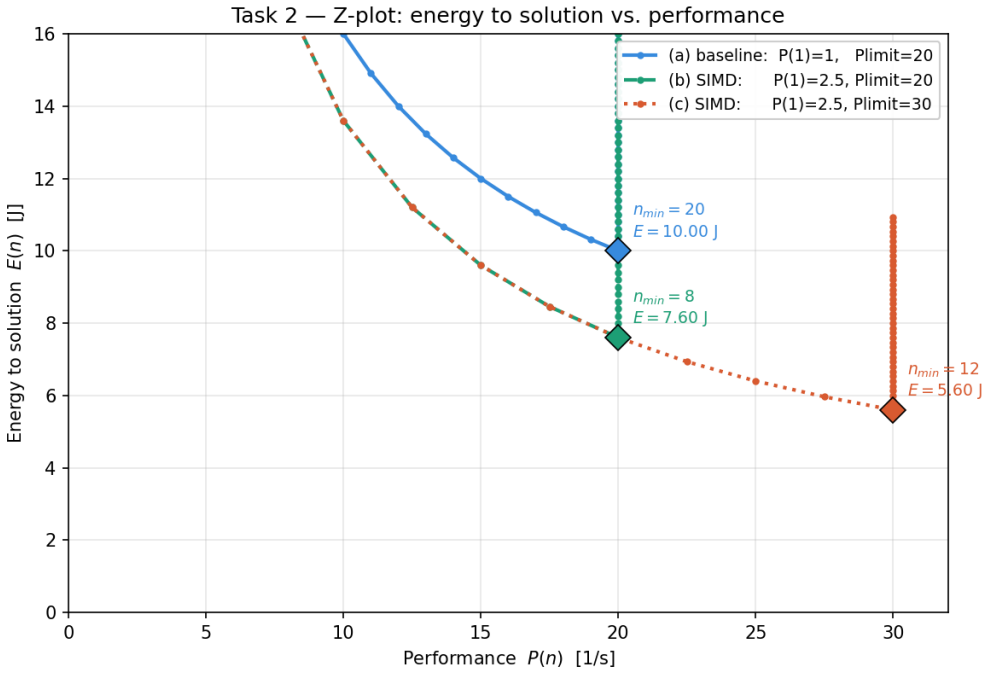


Figure 1: Z-plot of energy to solution $E(n)$ versus performance $P(n)$ for the three datasets of Q2(a–c). The number of cores n is the parameter along each curve; diamonds mark the energy minimum at the saturation knee. The vertical branch above each knee is the saturated regime ($n > n_{\min}$), where P is pinned at P_{limit} but energy keeps rising.

(d) Pure core-bound workload, $P_{\text{limit}} = \infty$ (15 crd)

Performance linear in f and chip power $W = W_0 + n\,W_2 f^2$ (i.e. per-core dynamic power $W_d = W_2 f^2$). With $P \propto nf$ and $T \propto 1/(nf)$:

$$E(f) \propto \frac{W_0 + n\,W_2 f^2}{nf} = \frac{1}{n}\frac{W_0}{f} + W_2 f.$$

Setting $dE/df = 0$:

$$-\frac{W_0}{nf^2} + W_2 = 0 \quad \Rightarrow \quad \boxed{f_{\text{opt}} = \sqrt{\frac{W_0}{n\,W_2}}.}$$

Interpretation.

- **Large** W_0 (baseline/idle power dominates) $\Rightarrow$ high f_{opt}: "race to idle" – finish fast to stop paying the constant baseline.
- **Small** $W_0 \Rightarrow f_{\text{opt}} \to 0$: run as slowly as possible – exactly the crude Q1(d) result ($E \propto f^2$) when only dynamic power matters.
- Also $f_{\text{opt}} \propto n^{-1/2}$: more active cores $\Rightarrow$ lower optimal per-core clock.

Q3: Peak performance and bandwidth (35 crd)

Chip: 2.0 GHz, 48 cores, AVX-512, 2 DP FMA/cycle/core; memory 12-channel DDR4-3200.

(a) Double-precision peak performance (5 crd)

With $n_{\text{SIMD}} = 512/64 = 8$, $n_{\text{FMA}} = 2$, $n_{\text{super}}^{FP} = 2$:

$$P_{\text{core}} = n_{\text{super}}^{FP} \cdot n_{\text{FMA}} \cdot n_{\text{SIMD}} \cdot f = 2 \cdot 2 \cdot 8 \cdot 2.0 \text{ GHz} = 64 \text{ Gflop/s},$$

$$\boxed{P_{\text{chip}} = 48 \times 64 = 3072 \text{ Gflop/s} = 3.07 \text{ Tflop/s}.}$$

(b) Theoretical memory bandwidth (5 crd)

DDR4-3200 = 3200 MT/s per channel, 8 B per transfer:

$$\boxed{b_S = 12 \times 3.2 \text{ GT/s} \times 8 \text{ B} = 307.2 \text{ Gbyte/s}.}$$

(c) STREAM Triad with nontemporal stores (20 crd)

Loop `a[i]=b[i]+s*c[i]`, $N = 10^9$. With NT stores there is no write-allocate: per iteration 2 loads (b, c) +1 NT store (a) = 24 B and 2 flops (one FMA).

Arithmetic time.

$$T_{\text{arith}} = \frac{2 \times 10^9 \text{ F}}{3.072 \times 10^{12} \text{ F/s}} \approx 0.65 \text{ ms}.$$

Data-transfer time.

$$T_{\text{mem}} = \frac{24 \times 10^9 \text{ B}}{307.2 \times 10^9 \text{ B/s}} \approx 78.1 \text{ ms}.$$

Since $T_{\text{mem}}/T_{\text{arith}} \approx 120$, the loop is strongly **memory-bound**; the memory bandwidth – not the FP units – is the relevant bottleneck. Equivalently, the code balance $B_c = 24\,\text{B}/2\,\text{F} = 12$ B/F gives a roofline limit $b_S/B_c = 25.6$ Gflop/s $\ll 3072$ Gflop/s.

Is the peak-performance bottleneck assumption justified? No. The FP units are oversized for this kernel by $\sim 120\times$, so assuming compute could limit it is unrealistic. To make the model realistic one would: use the *achievable* (measured/STREAM) bandwidth instead of the theoretical value (typically 70–80% of it, due to refresh, protocol and read/write turnaround); account for the fact that a single core cannot saturate the bus (bandwidth saturates only with enough cores); and verify that the compiler actually emits NT stores. The realistic performance predictor is the bandwidth limit $P = b_S/B_c$, not the peak.

(d) Speed-up on an NVIDIA A100-SXM4 (10 crd)

For a memory-bound kernel the speed-up equals the bandwidth ratio. With the A100 theoretical bandwidth $b_S \approx 1555$ GB/s (measured ≈ 1400 GB/s):

$$\boxed{\frac{1555}{307.2} \approx 5.1\times \quad (\approx 4.6\times \text{ using measured 1400 GB/s}).}$$

Q4: Energy of π (20 crd)

Hardware and method. Measurements were taken on **Fritz** (NHR@FAU) [**?**], one Intel Xeon Platinum 8360Y (Ice Lake-SP, 36 cores @ 2.4 GHz, AVX-512). A compute-bound π computation (numerical integration of $4/(1+x^2)$ on $[0, 1]$) was measured with `likwid-perfctr` on core 0 using the `ENERGY` group; reported energy is the whole-chip package energy `PWR_PKG_ENERGY`. Problem size $N = 2 \times 10^9$ for all cases.

Build. The scalar binary was built with `gcc -fno-tree-vectorize` (the Intel `icx` compiler vectorised the loop even with `-no-vec`); the AVX-512 binary with `icx`:

```
gcc -O3 -fno-tree-vectorize -march=icelake-server pi.c -o pi_scalar
icx -O3 -xICELAKE-SERVER -qopt-zmm-usage=high -fp-model=fast pi.c -o pi_avx512
```

Hardware counters were enabled with the Slurm option `-constraint=hwperf`; fixed clocks were set via `srun -cpu-freq` (`likwid-setFrequencies` is unavailable on the clusters), and "Turbo Mode" is the default with no frequency cap.

Results (six variants).

Frequency	SIMD	Runtime [s]	Power [W]	**Energy [J]**
Turbo (3.5 GHz)†	scalar	2.29	88.8	203.5
Turbo (3.4 GHz)†	AVX-512	1.19	103.3	123.1
2.4 GHz	scalar	3.34	99.2	331.9
2.4 GHz	AVX-512	1.68	100.1	168.0
1.2 GHz	scalar	6.69	56.1	375.4
1.2 GHz	**AVX-512**	3.36	55.7	**123.1**

† Turbo settles below the 3.5 GHz single-core nominal: 3500 MHz (scalar) vs. 3400 MHz (AVX-512), reflecting AVX clock throttling.

Least energy. Two variants tie at ≈123 J: *Turbo + AVX-512* (123.10 J) and 1.2 *GHz + AVX-512* (123.12 J), i.e. identical within measurement noise. **AVX-512 at 1.2 GHz consumed the least energy (123.1 J), tied with Turbo + AVX-512.**

Discussion.

- **SIMD wins at every frequency.** The AVX-512 build retires ∼6.5× fewer instructions (2.75 G vs. 18.0 G in `INSTR_RETIRED`) because eight DP lanes are packed per instruction. It finishes in roughly half the time at comparable power, so it uses far less energy. Vectorisation is the dominant lever.
- **Frequency: lower is better, but only down to a point.** For the AVX-512 builds, energy follows the $E \propto f^2$ law of Q1(d): 2.4 GHz (168 J) → 1.2 GHz (123 J), because dynamic power drops as f^3 while runtime only grows as $1/f$. The scalar builds, however, get *worse* at 1.2 GHz (331.9 → 375.4 J).
- **Why scalar breaks the f^2 rule: baseline power.** This is the Q2(d) effect in practice. Total power is not purely dynamic; a constant baseline W_0 (uncore, memory controller, leakage) is always paid. Halving the clock more than doubles the scalar runtime (3.34 → 6.69 s); over that long runtime the chip keeps paying W_0. Package power falls to 56 W but the runtime penalty wins, so energy rises – the "race-to-idle" regime where f_{opt} moves *up*, not down.
- **AVX-512 sits near the sweet spot.** Because vectorisation slashes the active runtime, the baseline-power penalty of running slowly is small, so the f^2 effect dominates and 1.2 GHz ties Turbo for the minimum.

Take-away. Vectorise first (the largest energy saver); then, for compute-bound code, lowering the clock helps through $E \propto f^2$ – but only until the constant baseline power W_0 makes the longer runtime cost more than the lower power saves. That crossover is exactly why the scalar variant worsens at 1.2 GHz while the AVX-512 variant improves.